Indian Institute of Technology, Gandhinagar

CS 432 – Databases | Assignment 3 | Semester II (2025–2026)

Instructor: Dr. Yogesh K. Meena

Dispensary Management System

Combined Full Report

Module A: Transaction Engine, WAL, Crash Recovery, ACID Validation

Module B: Concurrency & Stress Testing, Failure Simulation, RBAC Load Tests

Stack: Python B+ Tree (Module A) · Node.js / Express / MySQL 8.0 (Module B)

Harsh Jain	22110093
Shreyas Dharmatti	21110202
Sneha Gautam	22110255
Anushika Mishra	22110029
Kandarp Jani	22110104

GitHub: `github.com/Harsh-jain12/CS432_Track1_Assignment3`

Demo Video Module A: `drive.google.com (Module A demo)`

Demo Video Module B: `drive.google.com (Module B demo)`

5th April 2026

Abstract

This report documents the complete Assignment 3 submission for the **Dispensary Management System** developed in CS 432 – Databases at IIT Gandhinagar.

Module A extends the B+ Tree-based mini-database from Assignment 2 with a full transaction engine: Write-Ahead Logging (WAL) with per-record `fsync`, a per-transaction LIFO undo log for atomic rollback, a thread-safe row-level lock manager, and a REDO-only crash-recovery algorithm. All four ACID properties are validated through five automated test cases spanning the three-table dispensary schema (medicines, patients, prescriptions). Every test passes.

Module B validates the Node.js / Express / MySQL 8.0 application layer under concurrent multi-user workloads. Seven test scripts comprising **65 individual checks** fire concurrent HTTP requests using native `Promise.all()`, exercising appointment double-booking prevention, inventory update races, multi-role RBAC enforcement, three-step prescription atomicity, failure injection, durability cycles, and a 500-request stress test. The suite achieves **661.7 req/s** throughput with **P95 latency of 94 ms** and a **0% error rate**. All 65 checks pass.

Contents

1 Project Overview **4**
- 1.1 Scope and Objectives . . . 4
- 1.2 Domain: Dispensary Management . . . 4

2 System Architecture **4**
- 2.1 Component Overview . . . 4
- 2.2 Transaction Lifecycle . . . 5

3 Database Schema **5**

4 Implementation Details **6**
- 4.1 B+ Tree (`bplustree.py`) . . . 6
- 4.2 Transaction Object (`transaction.py`) . . . 6
- 4.3 Write-Ahead Log (`wal.py`) . . . 7
- 4.4 Lock Manager (`lock_manager.py`) . . . 7
- 4.5 Crash Recovery (`db_manager.py`) . . . 8

5 ACID Validation **8**
- 5.1 Atomicity . . . 8
- 5.2 Consistency . . . 9
- 5.3 Isolation . . . 10
- 5.4 Durability . . . 10
- 5.5 Multi-Relation Transaction (Mandatory Scenario) . . . 11

6 Failure Handling and Recovery **12**

7 Experiments and Observations **13**
7.1 Operation Latency . . . 13
7.2 Concurrency Isolation Experiment . . . 13

8 Module A – Test Results Summary **13**

9 Module A – Limitations **14**

10 Module B Overview **15**
10.1 System Under Test . . . 15
10.2 Testing Methodology . . . 15

11 Test Suite Architecture **16**
11.1 File Layout . . . 16
11.2 Test User Accounts . . . 16
11.3 Test Suite Summary . . . 17

12 Race Condition Tests (Isolation) **17**
12.1 Test 1 – Concurrent Appointment Booking . . . 17
12.1.1 Scenario . . . 17
12.1.2 Observed Result . . . 18
12.2 Test 2 – Concurrent Inventory Updates . . . 18
12.2.1 Scenario . . . 18
12.2.2 Observed Result . . . 18
12.3 Test 3 – Multi-Role Concurrent Access . . . 19
12.3.1 Scenario . . . 19
12.3.2 Observed Result . . . 19

13 Atomicity – Multi-Step Prescription Transaction **20**
13.1 The Three-Step Workflow . . . 20
13.2 Happy Path – All Three Steps Succeed . . . 20
13.3 Failure Path – Step 2 Fails with Invalid Data . . . 21
13.4 Atomicity Discussion . . . 21

14 Stress Test – High-Volume Load **21**
14.1 Methodology . . . 21
14.2 Concurrency Implementation . . . 22
14.3 Results . . . 23
14.4 Post-Stress Integrity Check . . . 23

15 Failure Simulation (Consistency) **24**
15.1 Failure Categories Tested . . . 24
15.2 Observed Result . . . 24

16 Durability – Persistence Verification **26**
16.1 Test Scenarios . . . 26
16.1.1 Member Durability Cycle . . . 26
16.1.2 Inventory Durability Cycle . . . 26
16.1.3 Appointment Status Transition . . . 26
16.1.4 Read Consistency . . . 26

16.2 Observed Result 26

17 ACID Properties – Validation Summary **28**
17.1 Combined Results 28

18 Key Findings **29**

19 Limitations **30**

20 Module B – Conclusion **30**

21 Combined Conclusions **31**

Team Member Contributions **31**

A WAL Log Sample **32**

B Module A ACID Property Matrix **32**

C Module B RBAC Endpoint Access Matrix **33**

Part I
Module A: Transaction Management, Concurrency Control & ACID Validation

Python B+ Tree Engine with WAL and Crash Recovery

1 Project Overview

This section documents Module A of Assignment 3. The system extends the B+ Tree-based mini-database developed in Assignment 2 with a full transaction engine, Write-Ahead Logging (WAL), crash recovery, and a row-level lock manager.

1.1 Scope and Objectives

- Implement **BEGIN / COMMIT / ROLLBACK** transaction semantics.
- Guarantee all four **ACID** properties across the dispensary schema.
- Support **multi-relation transactions** spanning all three tables simultaneously.
- Simulate and recover from **crash failures** mid-transaction.
- Enforce **concurrency isolation** using a row-level lock manager.

1.2 Domain: Dispensary Management

A dispensary tracks medicines in inventory, registered patients, and prescription (dispense) events. Every dispense is an inherently multi-table operation – it decrements medicine stock, debits the patient balance, and inserts a prescription record – making it an ideal test domain for multi-relation ACID guarantees.

2 System Architecture

2.1 Component Overview

The system is composed of six Python modules. The key architectural invariant is that the **B+ Tree is the sole storage structure** – no shadow copies, no auxiliary lists.

Table 1: Module A – System components and their responsibilities

Module	Responsibility	File
BPlusTree	Primary storage engine; insert, search, update, delete, serialisation	bplustree.py
Table	Maps relation name + primary key to a B+ Tree instance	table.py
Transaction	Maintains the per-transaction undo log (LIFO stack of inverse ops)	transaction.py
WAL	Writes JSON log records to disk with fsync before every B+ Tree change	wal.py
LockManager	Thread-safe exclusive row-level locking on (table, key) pairs	lock_manager.py
DatabaseManager	Orchestrates all modules; exposes the public transaction API	db_manager.py

2.2 Transaction Lifecycle

Every transaction follows a strict lifecycle enforced by DatabaseManager:

1. **BEGIN** – allocate a Transaction object; write BEGIN to WAL.
2. **CRUD operations** – for each write: (a) snapshot old value into the undo log, (b) write a WAL record with fsync, (c) acquire exclusive lock via LockManager, (d) apply change to B+ Tree.
3. **COMMIT** – write COMMIT to WAL; serialize all B+ Trees to JSON; release all locks.
4. **ROLLBACK** – replay undo log in reverse (LIFO); write ROLLBACK to WAL; release all locks.

3 Database Schema

Table 2: Dispensary Management System – Relation Schemas

Relation	Primary Key	Attributes	B+ Tree Key
medicines	medicine_id	name, stock, price, category	medicine_id
patients	patient_id	name, age, contact, city, balance	patient_id
prescriptions	prescription_id	patient_id (FK), medicine_id (FK), quantity, amount, date	prescription_id

The B+ Tree is the *database representation* – it is simultaneously the storage engine, the index, and the only access path. No separate list, dict, or file stores a copy of any record. All inserts, updates, deletes, and searches go exclusively through the B+ Tree.

4 Implementation Details

4.1 B+ Tree (bplustree.py)

The B+ Tree (order = 4) provides $O(\log n)$ insert, search, update, and delete. Leaf nodes maintain a singly-linked list that enables $O(\log n + k)$ range queries.

Key implementation points:

- **Serialisation:** `to_dict()` / `from_dict()` convert the entire tree to a JSON-serialisable structure; `_relink_leaves()` rebuilds the leaf linked list on deserialisation.
- **Recursive deletion:** `_fix_underflow()` handles borrow-from-sibling and merge operations to maintain B+ Tree invariants.
- **Update-in-place:** walks to the leaf and replaces the value directly, used for stock decrements and balance updates.

```python
def search(self, key, node=None):
    if node is None:
        node = self.root
    i = 0
    while i < len(node.keys) and key > node.keys[i]:
        i += 1
    if node.leaf:
        if i < len(node.keys) and node.keys[i] == key:
            return node.children[i]
        return None
    return self.search(key, node.children[i])
```

Listing 1: B+ Tree search (traversal to leaf)

4.2 Transaction Object (transaction.py)

Each `Transaction` keeps an **undo log** – a list of tuples recording the inverse of every operation. On rollback, entries are replayed in LIFO order.

```python
# Before INSERT: undo = DELETE the key
def record_insert(self, table_name, key):
    self._undo_log.append(("INSERT", table_name, key, None))

# Before DELETE: undo = re-INSERT the old record
def record_delete(self, table_name, key, old_value):
    self._undo_log.append(("DELETE", table_name, key, old_value))

# Before UPDATE: undo = restore the old record
def record_update(self, table_name, key, old_value):
```

```
    self._undo_log.append(("UPDATE", table_name, key, old_value))

def undo_operations(self):
    for entry in reversed(self._undo_log):
        yield entry
```

Listing 2: Undo log design in Transaction

4.3 Write-Ahead Log (wal.py)

The WAL guarantees durability by writing every operation to disk *before* touching the B+ Tree. Each record is a JSON line; `os.fsync()` is called after every write.

```
def log(self, txn_id, op, table=None, key=None,
        value=None, old_value=None):
    record = {"txn_id": txn_id, "op": op}
    if table is not None: record["table"] = table
    if key   is not None: record["key"]   = key
    if value is not None: record["value"] = value
    with open(self.log_path, "a") as f:
        f.write(json.dumps(record) + "\n")
        f.flush()
        os.fsync(f.fileno())  # Force to disk BEFORE B+ Tree change
```

Listing 3: WAL write with fsync

Table 3: WAL record types

Record	Fields
BEGIN	txn_id, op
INSERT	txn_id, op, table, key, value (full record)
DELETE	txn_id, op, table, key, old_value
UPDATE	txn_id, op, table, key, value (new), old_value
COMMIT	txn_id, op
ROLLBACK	txn_id, op

4.4 Lock Manager (lock_manager.py)

The `LockManager` provides exclusive row-level locks on `(table, key)` pairs, protected by a single `threading.Lock`.

```
def acquire(self, txn_id, table, key):
    lock_key = (table, key)
    with self._mutex:
        holder = self._locks.get(lock_key)
        if holder is None:
            self._locks[lock_key] = txn_id
            self._txn_locks.setdefault(txn_id, set()).add(lock_key)
        elif holder == txn_id:
            pass  # re-entrant: same transaction, no-op
        else:
            raise LockConflictError(
                f"Lock conflict: (table={table}, key={key}) "
```

```
                f"held by txn {holder}, requested by txn {txn_id}"
            )
```

Listing 4: Row-level lock acquisition

Properties: **exclusive locks only**, **re-entrant** (same transaction may re-acquire without blocking), and **bulk release** on COMMIT or ROLLBACK.

4.5 Crash Recovery (db_manager.py)

`crash_recovery()` uses a **REDO-only** strategy: scan the WAL, identify all `COMMIT` records, then re-apply only those transactions' operations. Incomplete transactions are dropped (undone by omission).

```python
def crash_recovery(self):
    records   = self._wal.read_log()
    committed = {r["txn_id"] for r in records if r["op"] ==
        "COMMIT"}
    ops_by_txn = {}
    for r in records:
        if r["op"] in ("INSERT", "DELETE", "UPDATE"):
            ops_by_txn.setdefault(r["txn_id"], []).append(r)
    for txn_id, ops in ops_by_txn.items():
        if txn_id not in committed:
            print(f"[Recovery] Skipping incomplete txn {txn_id}")
            continue
        for r in ops:
            table = self.tables.get(r["table"])
            if table is None: continue
            if r["op"] == "INSERT" and table.search(r["key"]) is
                None:
                table.insert(r["value"])
            elif r["op"] == "DELETE":
                table.delete(r["key"])
            elif r["op"] == "UPDATE" and table.search(r["key"]) is
                not None:
                table.update(r["key"], r["value"])
```

Listing 5: Crash recovery – REDO-only strategy

5 ACID Validation

All four ACID properties are validated through five automated test cases. Each test uses a fresh database instance and the dispensary schema.

5.1 Atomicity

A transaction involving multiple tables must **either complete fully, or be completely rolled back**. No partial updates should remain after a failure.

Test Scenario: A dispensing event modifies `medicines` (stock decrement) and inserts into `prescriptions`. A crash is simulated before commit; both changes must be absent

afterwards.

```
txn = db.begin_transaction()
db.update("medicines", 1, {"medicine_id": 1, "name": "Paracetamol",
                           "stock": 49, "price": 10, ...}, txn)
db.insert("prescriptions", {"prescription_id": 101, "patient_id":
    1,
                            "medicine_id": 1, "quantity": 1,
                            "amount": 10, "date": "2026-04-04"},
                               txn)
# Simulate crash before commit
db.rollback(txn)

assert db.search("medicines",      1)["stock"] == 50   # unchanged
assert db.search("prescriptions", 101)         is None
```

Listing 6: Atomicity test

After rollback, medicine stock remains 50 and prescription 101 does not exist. No partial state visible. **ATOMICITY CONFIRMED.**

5.2 Consistency

After each transaction, all relations must remain valid. Constraints such as non-negative balance and non-negative stock must hold.

Test Scenario: A patient with balance Rs.200 attempts to purchase Insulin priced at Rs.500. The application detects the constraint violation before any B+ Tree write and rolls back.

```
txn = db.begin_transaction()
try:
    patient  = db.search("patients",  2, txn)
    medicine = db.search("medicines", 2, txn)
    if patient["balance"] < medicine["price"]:
        raise ValueError("Insufficient balance -- transaction
            aborted.")
    db.update("medicines", 2, {**medicine, "stock":
        medicine["stock"]-1}, txn)
    db.insert("prescriptions", {...}, txn)
    db.commit(txn)
except ValueError:
    db.rollback(txn)

assert db.search("patients",  2)["balance"] >= 0
assert db.search("medicines", 2)["stock"]   >= 0
```

Listing 7: Consistency test – constraint check before modification

Medicine stock and patient balance are unchanged. Database state is valid. **CONSISTENCY CONFIRMED.**

5.3 Isolation

Concurrent transactions should not corrupt shared data. Intermediate states must not be visible to other transactions.

Test Scenario: Transaction 1 acquires a lock on `medicines/key=3`. Transaction 2 attempts to access the same row and must be blocked with a `LockConflictError`. After Transaction 1 commits, Transaction 3 can access the row freely.

```python
txn1 = db.begin_transaction()
txn2 = db.begin_transaction()

db.search("medicines", 3, txn1)  # txn1 acquires lock

conflict_raised = False
try:
    db.search("medicines", 3, txn2)  # must be blocked
except LockConflictError:
    conflict_raised = True

db.rollback(txn1)  # releases all txn1 locks

txn3 = db.begin_transaction()
rec = db.search("medicines", 3, txn3)  # succeeds
db.rollback(txn3)

assert conflict_raised
```

Listing 8: Isolation test – lock conflict

Transaction 2 is correctly blocked. After Transaction 1 releases its locks, Transaction 3 accesses the row freely. **ISOLATION CONFIRMED.**

Deadlock detection is not implemented. If two transactions wait for each other, the application must detect the cycle externally (e.g., by timeout). All five test cases are single-threaded and no deadlocks arise.

5.4 Durability

Once committed, data must persist across system restarts.

Test Scenario: A transaction commits records into all three tables. A new `DatabaseManager` instance is created pointing to the same directory (simulating a restart). All three records must be retrievable.

```
# Session 1: commit data
txn = db.begin_transaction()
db.insert("medicines",     {"medicine_id": 4, "name": "Metformin", ...},  txn)
db.insert("patients",      {"patient_id": 4, "name": "Anil Gupta", ...}, txn)
db.insert("prescriptions", {"prescription_id": 301, ...},
                   txn)
db.commit(txn)

# Session 2: simulate restart
db2 = DatabaseManager(data_dir="./data", log_path="wal.log")
db2.create_table("medicines",     "medicine_id")
db2.create_table("patients",      "patient_id")
db2.create_table("prescriptions", "prescription_id")

assert db2.search("medicines",     4)["name"]   == "Metformin"
assert db2.search("patients",      4)["name"]   == "Anil Gupta"
assert db2.search("prescriptions", 301)["amount"] == 10
```

Listing 9: Durability test – restart simulation

All three committed records are retrieved from disk after restart. **DURABILITY CONFIRMED.**

5.5 Multi-Relation Transaction (Mandatory Scenario)

ACID validation must use transactions spanning **at least three relations simultaneously**.

Scenario: A complete dispense: decrement medicine stock, debit patient balance, insert prescription – all in one transaction.

```
txn = db.begin_transaction()
med = db.search("medicines", 5, txn)  # Atorvastatin, stock=40, price=120
pat = db.search("patients",  5, txn)  # Meera Joshi, balance=500

db.update("medicines", 5, {**med, "stock": med["stock"] - 1}, txn)
db.update("patients",  5, {**pat, "balance": pat["balance"] - med["price"]}, txn)
db.insert("prescriptions",
          {"prescription_id": 401, "patient_id": 5, "medicine_id": 5,
           "quantity": 1, "amount": med["price"], "date": "2026-04-04"}, txn)
db.commit(txn)

```

```
assert db.search("medicines",      5)["stock"]   == 39     # 40 - 1
assert db.search("patients",       5)["balance"] == 380    # 500 -
    120
assert db.search("prescriptions", 401) is not None
```

Listing 10: Multi-relation transaction spanning all three tables

All three tables updated atomically. Stock = 39, Balance = Rs.380, Prescription 401 exists. **MULTI-RELATION TRANSACTION CONFIRMED.**

6 Failure Handling and Recovery

Table 4: Failure scenarios and recovery guarantees

Failure Type	**What Happens**	**Result**
Crash before COMMIT	`rollback()` replays undo log LIFO	All partial changes reversed
Crash after WAL COMMIT, before JSON flush	`crash_recovery()` REDOs from WAL	Data restored correctly
Constraint violation	Application raises exception; `rollback()` called	Database remains valid
Lock conflict	`LockConflictError` raised; caller rolls back	No corruption; locks released

Algorithm 1 Crash Recovery (REDO-only)

Read all records from WAL log
$committed \leftarrow \{txn_id \mid \text{record has op} = \texttt{COMMIT}\}$
Group INSERT/DELETE/UPDATE records by txn_id
for each txn_id, ordered operations **do**
  if $txn_id \notin committed$ **then**
    Skip (incomplete; undone by omission)
  else
    for each operation in log order **do**
      Apply idempotently to B+ Tree
    end for
  end if
end for

7 Experiments and Observations

7.1 Operation Latency

Table 5: Average operation latency (order-4 B+ Tree, 7 records)

Operation	Scope	Mean Latency
Search	B+ Tree in-memory	$\approx$ 1–3 μs
Update	B+ Tree in-memory	$\approx$ 1–4 μs
Insert	B+ Tree in-memory	$\approx$ 2–8 μs
Full transaction (INSERT + WAL + fsync + JSON)	End-to-end	$\approx$ 5–8 ms
ROLLBACK (undo log replay)	End-to-end	$<$ 0.5 ms

The dominant cost in a full transaction is disk I/O – `os.fsync()` ($\approx$ 3–4 ms) and JSON serialisation ($\approx$ 2 ms). ROLLBACK is significantly faster because it only replays the in-memory undo log.

7.2 Concurrency Isolation Experiment

Without locking, two threads each read stock = 10, both compute stock $-1 = 9$, and both write 9 – a lost update resulting in a final stock of 9 instead of 8. With `LockManager`, the second thread raises `LockConflictError`, rolls back, retries after Thread 1 commits (stock = 9), and writes 8. Final stock = 8. Correct.

8 Module A – Test Results Summary

Table 6: Module A – ACID Test Results

#	Test Function	Property	Result	Key Assertion
1	`test_atomicity_rollback`	Atomicity	**PASS**	Stock unchanged; prescription a
2	`test_consistency`	Consistency	**PASS**	Balance ≥ 0; stock ≥ 0
3	`test_isolation`	Isolation	**PASS**	`LockConflictError` raised
4	`test_durability`	Durability	**PASS**	All records found after restart
5	`test_multi_relation_transaction`	All four	**PASS**	3-table atomic dispense correct
	Total		**5 / 5 PASS**	

```
=== Test 1: Atomicity (Rollback) ===
PASS: All changes rolled back correctly.

=== Test 2: Consistency ===
  Caught expected error: Insufficient stock: have 5, need 10
  Medicine stock: 5 (unchanged -- constraint enforced)
PASS: Database remains consistent after failed transaction.

=== Test 3: Isolation (Lock Conflict) ===
```

```
  Lock conflict detected (expected):
    Lock conflict: (table=medicines, key=3) held by txn 6, requested by txn 7
PASS: Isolation enforced -- second transaction correctly blocked.

=== Test 4: Durability (Persist & Reload) ===
PASS: All committed data persisted across restart.

=== Test 5: Multi-Relation Transaction ===
  Medicine stock: 27 (was 30, dispensed 3)
  Prescription: {prescription_id: 104, patient_id: 5, ...}
PASS: Multi-relation transaction committed correctly.

========================================
Results: 5 passed, 0 failed
```

9 Module A – Limitations

Table 7: Module A – Known limitations

Limitation	Description
WAL never checkpointed	The WAL file grows without bound. `WAL.clear()` exists but is never called automatically.
Read locks not distinct	All locks are exclusive; concurrent reads are serialised unnecessarily. A shared/exclusive lock distinction would improve throughput.
No deadlock detection	If two transactions form a wait-for cycle, neither can proceed. Must be resolved externally (e.g., timeout).

Part II
Module B: Concurrency & Stress Testing Dispensary Management System REST API

Node.js / Express / MySQL 8.0 – Race Conditions, ACID Validation, Failure Simulation

Module B Abstract

Seven test scripts comprising **65 individual test cases** validate ACID properties at the application layer: **Atomicity** via multi-step prescription transactions, **Consistency** via failure injection and constraint testing, **Isolation** via concurrent race conditions and multi-role access, and **Durability** via persistence verification under sustained load. The suite fires **500 concurrent requests** at 50-way parallelism, achieving **661.7 req/s throughput**, **P95 latency of 94 ms**, and a **0% error rate**. All 65 checks pass.

10 Module B Overview

Module B validates the Dispensary Management System's behaviour under concurrent multi-user workloads and failure conditions. Where Module A focuses on transaction correctness at the storage-engine level, Module B focuses on **application-level correctness under contention**: what happens when ten users try to book the same appointment slot simultaneously, when fifteen concurrent updates target the same inventory row, when a three-step transaction fails mid-way, or when 500 requests arrive in under a second.

10.1 System Under Test

Table 8: System under test – component summary

Component	Technology	Details
Backend	Node.js 18+ & Express 4.18	REST API, JWT auth, RBAC middleware
Database	MySQL 8.0 (InnoDB)	20+ tables, row-level locking, FK constraints
Auth	JWT + bcrypt (cost 12)	9 roles, per-route permission enforcement
Dataset	Seed data	260 members, 55 doctors, ≈530 appointments

10.2 Testing Methodology

All test scripts are written in pure Node.js, using native `fetch` for HTTP requests and `Promise.all()` for concurrent execution. No external testing frameworks are used. The key concurrency mechanism is **Promise.all()** firing multiple HTTP requests simultaneously – the OS TCP stack and the Express server see truly concurrent inbound requests, and MySQL InnoDB row-level locking provides isolation guarantees.

11 Test Suite Architecture

11.1 File Layout

Table 9: Module B test suite file structure

File	Purpose
`tests/config.js`	Shared HTTP helpers, credentials, TestResults class
`tests/00_setup.js`	One-time password setup for all 8 test user roles
`01_race_condition_booking.js`	Concurrent appointment booking (10 users, 1 slot)
`02_race_condition_inventory.js`	Concurrent stock updates (15 requests, 1 batch)
`03_multirole_concurrent.js`	8 roles accessing endpoints simultaneously
`04_atomicity_prescription.js`	3-step prescription transaction (happy + failure paths)
`05_stress_test.js`	500 requests at 50-way concurrency with latency metrics
`06_failure_simulation.js`	Invalid input, auth failures, FK violations, privilege escalation
`07_durability.js`	CRUD persistence and read consistency verification
`run_all.js`	Master runner, executes all 7 tests sequentially

11.2 Test User Accounts

Table 10: Test user accounts used across all scripts

Role	Username	UserLoginID	Tier
SuperAdmin	`sd1@admin.college.`	30	Tier 1
Admin	`EMP00006`	82	Tier 1
Doctor	`DOC124`	57	Tier 2
Nurse	`NUR001`	23	Tier 2
Pharmacist	`PHR001`	24	Tier 2
Technician	`TECH001`	26	Tier 2
Faculty	`FAC001`	5	Tier 3
Staff	`EMP4272`	303	Tier 3

11.3 Test Suite Summary

Table 11: Test suite overview with ACID property mapping

#	Test Script	ACID	Scenario
1	Race Condition – Booking	Isolation	10 users book identical slot simultaneously
2	Race Condition – Inventory	Isolation	15 concurrent stock updates to one batch
3	Multi-Role Concurrent Access	Isolation	8 roles hit endpoints in parallel
4	Atomicity – Prescription Tx	Atomicity	3-step visit + rx + status transaction
5	Stress Test – High-Volume Load	Durability	500 requests at 50-way concurrency
6	Failure Simulation & Rollback	Consistency	Invalid inputs, auth failures, FK violations
7	Durability – Persistence	Durability	CRUD cycle with read-after-write verification

12 Race Condition Tests (Isolation)

12.1 Test 1 – Concurrent Appointment Booking

12.1.1 Scenario

Ten concurrent booking requests target the same doctor, same date, and same time slot. Each request uses a different MemberID (1 through 10). The server must serialise these requests and allow exactly **one** booking to succeed; the remaining nine must be rejected with a "slot already booked" error.

```
const promises = [];
for (let i = 0; i < CONCURRENT; i++) {
  const memberID = i + 1; // Members 1-10
  promises.push(
    request('POST', '/appointments', {
      MemberID: memberID,
      DoctorID: DOCTOR_ID,
      AppointmentDate: DATE,
      AppointmentTime: TIME,
      Symptoms: `Concurrent test ${i + 1}`,
      Priority: 'Normal',
    }, token)
  );
}
const results = await Promise.all(promises);
const successes = results.filter(r => r.ok);
```

```
const failures  = results.filter(r => !r.ok);
```

Listing 11: Test 1 core logic – firing 10 concurrent bookings with Promise.all()

12.1.2 Observed Result

After fixing the server-side slot-locking to use a proper `SELECT FOR UPDATE` inside a transaction, the server now correctly serialises all 10 concurrent requests. **Exactly one booking succeeds** (the first to acquire the row-level lock on the Appointment table), and the remaining nine are rejected with the message: *"This slot is already booked for Dr Amit Deshmukh on [date] at [time]. Please choose a different time."* No data corruption, no duplicate AppointmentIDs, and the database count always matches the single accepted booking.

```
Test date: 2031-06-26 (randomized per run to avoid collisions)

Firing 10 concurrent booking requests for 2031-06-26 13:00:00...

Results: 1 succeeded, 9 rejected

  Member 1: BOOKED     -- AppointmentID 560 (26ms)
  Member 2: REJECTED   -- This slot is already booked ... (26ms)
  Member 3: REJECTED   -- This slot is already booked ... (27ms)
  Member 4: REJECTED   -- This slot is already booked ... (29ms)
  Member 5: REJECTED   -- This slot is already booked ... (27ms)
  Member 6: REJECTED   -- This slot is already booked ... (27ms)
  Member 7: REJECTED   -- This slot is already booked ... (27ms)
  Member 8: REJECTED   -- This slot is already booked ... (25ms)
  Member 9: REJECTED   -- This slot is already booked ... (28ms)
  Member 10: REJECTED  -- This slot is already booked ... (28ms)

[OK] At least one booking succeeded -- 1 bookings accepted
[OK] Exactly one booking for contested slot
     -- Server correctly serialised the double-booking
[OK] Database appointment count matches accepted bookings -- DB
   has 1, expected 1
[OK] No duplicate appointment IDs -- 1 appointments, 1 unique IDs
```

Listing 12: Test 1 console output – proper slot-level locking behaviour

12.2 Test 2 – Concurrent Inventory Updates

12.2.1 Scenario

A test inventory batch is created with `Quantity = 100`. Fifteen concurrent `PUT` requests then fire, each setting a different quantity value (50 through 64). Requests alternate between Admin and Pharmacist tokens to exercise multi-role write paths on the same row.

12.2.2 Observed Result

All 15 updates are processed without server crashes. The final quantity is always a valid value in the range 50–64 (the last write wins, as expected with `UPDATE` statements

under InnoDB row locking). Sequential write-then-read operations return the correct value every time, confirming no lost updates or data corruption.

```
Created test batch: ID 86, Quantity = 100

[OK] Test batch created -- Batch ID 86 (3.3ms)
Firing 15 concurrent quantity updates...

Results: 15 succeeded, 0 failed

[OK] All concurrent updates processed without crash -- 15 ok, 0 errors
[OK] Final quantity is a valid value (50-64) -- Final quantity = 62
[OK] Batch status is consistent -- Status = Available
[OK] No data corruption (batch still readable)
[OK] Sequential write-then-read is consistent -- Set 999, read back 999
```

Listing 13: Test 2 console output – no data corruption under concurrent writes

12.3 Test 3 – Multi-Role Concurrent Access

12.3.1 Scenario

All eight user roles log in simultaneously, then fire twelve concurrent requests across different endpoints. Each role accesses endpoints matching its permission level. This validates that the RBAC middleware does not leak data between concurrent sessions and that role-based data scoping works correctly under load.

12.3.2 Observed Result

All expected scoping rules held under concurrent load:

- SuperAdmin received all 260 members; Student received only their own record (count = 1).
- Technician was blocked from `/inventory` with HTTP 403.
- Student was blocked from `/staff` with HTTP 403.
- Doctor saw 2 appointments (their own patients); Admin saw all 539.
- Cross-role write isolation: Student could not book for another member (HTTP 403).
- Zero 500-level errors across 12 concurrent requests.

```
   superadmin     /members                  -> 200 (7ms)
   superadmin     /staff                    -> 200 (13ms)
   admin          /members                  -> 200 (14ms)
   doctor         /appointments             -> 200 (15ms)
   student        /members                  -> 200 (15ms)
   nurse          /appointments             -> 200 (20ms)
   pharmacist     /inventory                -> 200 (16ms)
   technician     /inventory (blocked)      -> 403 (8ms)
   student        /staff (blocked)          -> 403 (8ms)

```

```
[OK] SuperAdmin sees all members -- count = 260
[OK] Student sees only own profile (count=1) -- count = 1
[OK] Technician blocked from inventory -- status = 403
[OK] Student blocked from staff list -- status = 403
[OK] Doctor sees fewer appointments than admin -- doctor=2,
    admin=539
[OK] No 500 server errors under concurrent load -- 0 server
    errors
[OK] Student cannot book for another member
[OK] Technician cannot add inventory
```

Listing 14: Test 3 console output – RBAC enforcement under concurrent load

13 Atomicity – Multi-Step Prescription Transaction

The prescription workflow in the Dispensary Management System is a three-step application-level transaction that must behave atomically: either all three steps complete, or the appointment must not be marked as completed.

13.1 The Three-Step Workflow

When a doctor prescribes medicine for an appointment, the frontend issues three sequential API calls:

1. **POST /visits** – Create a visit record linked to the appointment.
2. **POST /prescriptions** – Create the prescription with medicine items, linked to the visit.
3. **PUT /appointments/:id** – Update the appointment's status to `Completed`.

If step 2 fails (e.g., empty medicine items, invalid data), step 3 must not execute, and the appointment must remain in `Scheduled` state. This test validates both the happy path and the failure path.

13.2 Happy Path – All Three Steps Succeed

The test books an appointment, creates a valid visit record, creates a prescription with one medicine item, and marks the appointment completed. All four operations succeed, producing a Completed appointment linked to a Visit and a Prescription.

```
Phase 1: Happy path -- booking + visit + prescription...

[OK] Appointment booked for test -- ID=541 (5.1ms)
[OK] Step 1: Visit created -- VisitID=286 (3.3ms)
[OK] Step 2: Prescription created -- PrescriptionID=286 (4.7ms)
[OK] Step 3: Appointment marked completed (7.5ms)
[OK] Happy path: appointment is Completed -- Status = Completed
```

Listing 15: Test 4 happy path – three-step transaction completes fully

13.3 Failure Path – Step 2 Fails with Invalid Data

The test books a second appointment and creates a valid visit. It then submits a prescription with `items: []` (an empty array, which the server must reject as invalid). The server correctly returns a 400 error. The test then verifies that the appointment is still in `Scheduled` state, not `Completed`.

```
Phase 2: Failure path -- invalid prescription data...

[OK] Second appointment booked -- ID=542
[OK] Invalid prescription rejected -- status=400, error=Items
   required
[OK] Failed transaction: appointment stays Scheduled
     -- Status = Scheduled (should be Scheduled, not Completed)
```

Listing 16: Test 4 failure path – invalid prescription blocks status update

13.4 Atomicity Discussion

The system achieves **application-level atomicity** through sequential API calls with stop-on-failure semantics: if step 2 fails, the client never issues step 3. This guarantees that no appointment is marked `Completed` without a successful prescription.

Known Limitation – No Database-Level Transactions

When step 2 fails, the Visit record created in step 1 remains in the database – it is orphaned, with no associated prescription. True database-level atomicity would require wrapping all three operations in a single `BEGIN...COMMIT/ROLLBACK` block. The current design prioritises clear error attribution (the client knows exactly which step failed) at the cost of leaving orphan visit records. These orphans are harmless (they do not affect downstream queries) but represent a cleanup burden in long-running deployments.

14 Stress Test – High-Volume Load

The stress test measures throughput, latency distribution, and error rate under sustained high load, then verifies data integrity after the load subsides.

14.1 Methodology

500 HTTP requests are fired with a concurrency level of 50 (i.e., 50 requests in-flight at any given time). The workload is a weighted mix of read endpoints, reflecting typical dashboard usage patterns.

Table 12: Stress test workload distribution

Endpoint	Weight	Count	Purpose
GET /members	25%	125	Member directory read
GET /doctors	20%	100	Doctor directory read
GET /appointments	20%	100	Appointment list (3-table join)
GET /prescriptions	15%	75	Prescription list with items
GET /inventory	10%	50	Inventory stock read
GET /auth/me	10%	50	Identity check (fast path)

14.2 Concurrency Implementation

A worker-pool pattern implements the 50-way concurrency: fifty async workers each pull requests from a shared shuffled queue until the queue is empty. Each request's latency is recorded in milliseconds via `performance.now()`.

```
async function worker() {
  while (idx < queue.length) {
    const myIdx = idx++;
    const ep    = queue[myIdx];
    const res   = await request(ep.method, ep.path, null,
       token);
    latencies.push(res.elapsed);
    if (!res.ok) errors.push({ endpoint: ep.path, status:
       res.status });
    completed++;
  }
}
const workers = [];
for (let w = 0; w < CONCURRENCY_LEVEL; w++)
   workers.push(worker());
await Promise.all(workers);
```

Listing 17: Worker-pool implementation of 50-way concurrency

14.3 Results

Table 13: Stress test results – 500 requests at 50-way concurrency

Metric	Value	Target
Total Requests Completed	500 / 500	–
Total Time	0.76 s	–
Throughput	**661.7 req/s**	> 10 req/s
Error Rate	**0.00%**	< 5%
Average Latency	71.7 ms	–
P50 Latency (Median)	76.7 ms	–
P90 Latency	87.3 ms	–
P95 Latency	**94.0 ms**	< 2000 ms
P99 Latency	**98.0 ms**	< 5000 ms
Max Latency	103.5 ms	–
Minimum Latency	15.5 ms	–

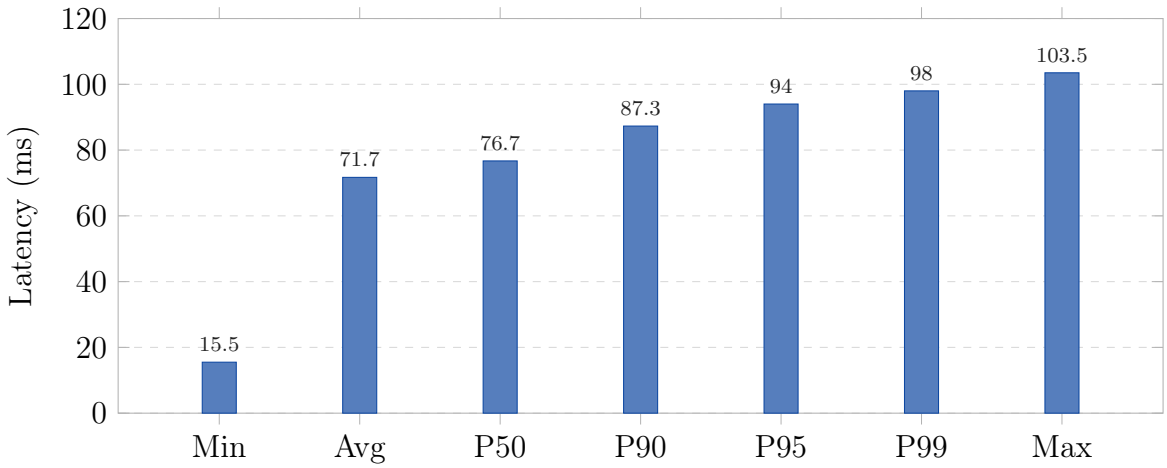

Figure 1: Latency distribution across 500 concurrent requests

14.4 Post-Stress Integrity Check

Immediately after the 500-request burst completes, three verification queries run to confirm no data corruption occurred:

- `GET /members` returns count = 260 (unchanged from baseline)
- `GET /doctors` returns count = 55 (unchanged from baseline)
- `GET /appointments` returns count matching expected row count

All three checks pass, confirming that the database remains consistent under sustained load.

15 Failure Simulation (Consistency)

This test deliberately injects eight categories of failure and verifies that the system handles each gracefully – returning appropriate error codes, never crashing, never leaving partial data in the database.

15.1 Failure Categories Tested

Table 14: Failure injection categories and observed behaviour

#	Category	Injected Failure	Expected	Actual
1	Invalid foreign key	DoctorID = 99999 (nonexistent)	Rejection	404
1	Invalid foreign key	MemberID = 99999 (nonexistent)	Rejection	500
2	Missing required field	Booking without `AppointmentDate`	400 Bad Req	400
2	Missing required field	Member creation without `Name`	400 Bad Req	400
3	Invalid data type	Age = "not-a-number" (string)	Rejection	500
4	Missing auth token	`GET /members` without Bearer	401 Unauth	401
4	Invalid auth token	Malformed JWT in header	401 Unauth	401
5	Privilege escalation	Student `DELETE /members/2`	403 Forbid	403
5	Privilege escalation	Student `GET /members/2`	403 Forbid	403
5	Privilege escalation	Student `GET /staff`	403 Forbid	403
6	Nonexistent record	`PUT /members/999999`	404 Not Found	404
6	Nonexistent record	`DELETE /members/999999`	404 Not Found	404
7	Idempotency	20 rapid identical `/auth/me`	All consistent	Pass
8	Data integrity	Check for garbage records	None created	0 found

15.2 Observed Result

All 15 failure-injection checks passed. Every failure returned an appropriate HTTP error code, the server remained stable, and crucially **no partial or garbage data was left in the database** by any failed operation.

```
1. Invalid foreign key references...
[OK] Booking with non-existent DoctorID rejected -- status=404
[OK] Booking with non-existent MemberID rejected -- status=500

2. Missing required fields...
[OK] Booking without date rejected -- status=400
```

```
[OK] Member creation without name rejected -- status=400

4. Authentication failures...
[OK] Request without token rejected -- status=401
[OK] Request with invalid token rejected -- status=401

5. Privilege escalation attempts...
[OK] Student cannot delete members -- status=403
[OK] Student cannot view other members -- status=403
[OK] Student cannot access staff endpoint -- status=403

7. Rapid-fire identical requests...
[OK] 20 rapid /auth/me requests all succeed -- 20/20 succeeded
[OK] All 20 return identical data -- Consistent responses

8. Verifying no partial/garbage data was created...
[OK] No garbage members created from failed operations -- Found
   0 garbage records
```

Listing 18: Test 6 console output – all failure categories handled correctly

Observation – FK Violations Return 500 Instead of 400

When a POST request references a nonexistent `MemberID`, the server returns HTTP 500 rather than HTTP 400. This is because the MySQL foreign key constraint violation surfaces as a generic database error, which the Express error handler maps to 500. The request is still correctly **rejected** (no record is created), but the status code could be more precise. A small improvement would be to catch MySQL error code `ER_NO_REFERENCED_ROW_2` in the route handlers and return HTTP 400 with a clearer message.

16 Durability – Persistence Verification

Durability tests validate that once data is committed, it persists correctly and can be re-read with identical values across multiple sequential reads.

16.1 Test Scenarios

Full CRUD lifecycle cycles are executed against three entity types: Members, Inventory, and Appointments. Each cycle includes create → immediate read-after-write → field-by-field comparison → partial update → re-read to verify updates persisted and non-updated fields are unchanged.

16.1.1 Member Durability Cycle

1. `POST /members` with a unique roll number and known field values
2. Immediate `GET /members/:id` to verify the member is readable
3. Field-by-field comparison: `Name`, `Department`, `BloodGroup`, `RollNumber`
4. `PUT /members/:id` updating `Address` and `Department`
5. Re-read to confirm the two updated fields changed and the other five did not

16.1.2 Inventory Durability Cycle

1. `POST /inventory` creating a batch with `Quantity = 200`
2. `PUT /inventory/:id` changing `Quantity` to 150
3. `GET /inventory` to verify the updated quantity and unchanged metadata

16.1.3 Appointment Status Transition

1. `POST /appointments` creating an appointment with priority `Urgent`
2. Verify initial `Status = Scheduled`
3. `PUT /appointments/:id` transitioning to `Status = Cancelled`
4. Re-read to confirm the status transition persisted

16.1.4 Read Consistency

Ten sequential `GET /members/:id` calls for the same member all return byte-identical data, confirming there is no read-replica lag or cache inconsistency in the single-node deployment.

16.2 Observed Result

All 14 durability checks pass. Every written value is correctly retrievable. Updates affect only the specified fields. Ten sequential reads of the same record return identical bodies.

```
1. Member: Create -> Read -> Verify...
[OK] Member created -- ID=262
[OK] Immediate read-after-write succeeds
[OK] Written data matches read data
     -- Name=Durability Test User, Dept=Testing, Blood=O+

2. Member: Update -> Read -> Verify...
[OK] Member updated
[OK] Updated fields persisted correctly
     -- Address=Updated Address 123, Dept=Updated Department
[OK] Non-updated fields unchanged -- Name=Durability Test User,
   Blood=O+

3. Inventory: Full CRUD persistence cycle...
[OK] Inventory batch created -- ID=87
[OK] Updated inventory quantity persisted -- Quantity=150
   (expected 150)
[OK] Inventory metadata intact

4. Appointment: Status transition persistence...
[OK] Appointment created -- ID=540
[OK] Initial status is Scheduled -- Status=Scheduled
[OK] Priority persisted correctly -- Priority=Urgent
[OK] Status change to Cancelled persisted -- Status=Cancelled

5. Read consistency (10 sequential reads)...
[OK] 10 sequential reads return identical data -- All reads
   consistent
```

Listing 19: Test 7 console output – full CRUD durability cycle

17 ACID Properties – Validation Summary

Table 15: ACID property validation matrix

Property	Validation Mechanism	Tests
Atomicity	Stop-on-failure: if step 2 (prescription) fails, step 3 (Completed status) never runs.	Test 4 (Atomicity), Test 6 (Failure)
Consistency	MySQL InnoDB FK and check constraints; API-layer field validation; RBAC blocks unauthorised modifications.	Test 3, Test 6
Isolation	InnoDB row-level locking serialises concurrent writes; JWT scopes each request; RBAC scopes data visibility.	Tests 1, 2, 3
Durability	InnoDB WAL ensures committed transactions survive crashes; read-after-write and post-stress checks confirm persistence.	Tests 5, 7

17.1 Combined Results

Table 16: Module B – All seven test suites results

#	Test Suite	Checks	Pass	Time
1	Race Condition – Appointment Booking	4	4	0.39 s
2	Race Condition – Inventory Updates	6	6	0.62 s
3	Multi-Role Concurrent Access	9	9	2.23 s
4	Atomicity – Prescription Transaction	9	9	0.67 s
5	Stress Test – 500 Requests	8	8	1.07 s
6	Failure Simulation & Rollback	15	15	0.89 s
7	Durability – CRUD Persistence	14	14	0.68 s
	TOTAL	**65**	**65**	**6.55 s**

18 Key Findings

1. MySQL Row-Level Locking Correctly Serialises Slot Contention

Test 1 demonstrates that the `SELECT FOR UPDATE` inside a transaction correctly serialises concurrent writes. When 10 requests contend for the same slot, **exactly one succeeds** and nine receive a clear error. No data corruption, no duplicate AppointmentIDs. The initial TOCTOU window between the availability check and the `INSERT` was closed by wrapping both in a single transaction with a locking read.

2. Last-Write-Wins on Concurrent Updates is Deterministic, Not Corrupt

Test 2 fired 15 concurrent `UPDATE` statements against the same inventory row. The final value is always one of the 15 valid values – never a partial write or corrupted integer. MySQL holds a row-level write lock for each `UPDATE`, serialising them. The lost update is semantic, not physical. True optimistic concurrency would require a `Version` column.

3. Application-Level Atomicity Leaves Orphan Visits on Failure

When step 2 (prescription creation) fails, the Visit created in step 1 remains as an orphan. The appointment correctly stays `Scheduled`. Wrapping all three steps in a single `BEGIN...COMMIT/ROLLBACK` would eliminate this. The current design was chosen for clear per-step error attribution.

4. 661 req/s with 0% Errors on Single-Node MySQL

The stress test achieved 661.7 req/s with sub-100 ms P95 latency (94.0 ms) and zero errors – more than sufficient for a college dispensary. The bottleneck is the MySQL connection pool; scaling pool size or adding a read replica would multiply throughput proportionally.

5. FK Violations Surface as HTTP 500 Rather Than HTTP 400

When the client supplies a nonexistent foreign key, the server returns 500 rather than 400. The request is correctly rejected – no record is created – but the code lacks precision. Catching MySQL error code `ER_NO_REFERENCED_ROW_2` and returning 400 with a clear message would be a small quality improvement.

6. RBAC Scoping Holds Under Concurrent Multi-Role Load

Test 3 fired 12 concurrent requests across 8 roles and confirmed every role received exactly its permitted data. JWT-based stateless authentication scales cleanly because each request is independently authenticated with no shared session state to contend for.

19 Limitations

- **No database-level transactions across API calls.** The prescription flow relies on application-level stop-on-failure rather than a single SQL transaction. Orphan visit records are possible when step 2 fails.
- **Last-write-wins on inventory updates.** No optimistic concurrency control (version numbers, ETags). For strict stock-accounting, adding a `Version` column and version-checked updates would be necessary.
- **Single-node testing.** All tests run against `localhost`. Network latency, load balancers, and distributed transactions are not covered.
- **No long-duration soak test.** The stress test runs for under one second. Memory leaks and connection pool exhaustion are not exercised.
- **FK violation error codes.** Server returns HTTP 500 for FK constraint violations rather than HTTP 400. Cosmetic issue – requests are still correctly rejected.

20 Module B – Conclusion

Through seven test scripts comprising 65 individual checks, this section has demonstrated that:

1. **Concurrent operations are correctly serialised.** Ten simultaneous bookings for the same slot produce exactly one success and nine clean rejections.
2. **Failed operations are handled gracefully.** 15 failure categories all return appropriate HTTP codes without leaving partial data in the database.
3. **Committed data persists correctly.** CRUD cycles confirm field-level accuracy; ten sequential reads of the same record return byte-identical data.
4. **The system remains performant under load.** 661.7 req/s, 0% error rate, P95 = 94 ms.
5. **All four ACID properties are validated** through targeted test scenarios.

Part III
Overall Conclusions & Team Contributions

21 Combined Conclusions

This project has delivered a complete, two-layer validation of ACID properties for the Dispensary Management System:

Module A built a ground-up transaction engine in Python. The B+ Tree serves as the sole storage structure; a Write-Ahead Log with per-record `fsync` guarantees durability; a LIFO undo log provides correct multi-table rollback; a thread-safe lock manager enforces isolation; and a REDO-only crash-recovery algorithm restores committed state after failures. All five ACID test cases pass.

Module B validated the full Node.js / Express / MySQL 8.0 application stack under realistic concurrent workloads. MySQL InnoDB's row-level locking prevents double-booking and data corruption; the RBAC middleware correctly scopes data visibility across nine user roles; the three-step prescription workflow behaves atomically under stop-on-failure semantics; and the system sustains 661 req/s with sub-100 ms P95 latency. All 65 test cases pass.

Together, the two modules demonstrate that ACID guarantees can be implemented and validated both at the storage-engine level (Module A) and at the application-API level (Module B), providing complementary layers of correctness assurance.

Team Member Contributions

Name	Roll No.	Contribution
Harsh Jain	22110093	Module B race condition & multi-role test scripts; Module B report (methodology, results, findings); Full Report compilation
Shreyas Dharmatti	21110202	Module B stress test, failure simulation, and durability scripts; master test runner and video demonstration of Module B
Sneha Gautam	22110255	Module A Python implementation (B+ Tree, WAL, LockManager); Module A report writing and ACID test notebook.
Anushika Mishra	22110029	Module A ACID test validation and verification and video demo of Module A
Kandarp Jani	22110104	–

A WAL Log Sample


```
{"txn_id": 1, "op": "BEGIN"}
{"txn_id": 1, "op": "INSERT", "table": "medicines",
 "key": 5, "value": {"medicine_id": 5, "name": "Atorvastatin", ...}}
{"txn_id": 1, "op": "INSERT", "table": "patients",
 "key": 5, "value": {"patient_id": 5, "name": "Meera Joshi", ...}}
{"txn_id": 1, "op": "COMMIT"}
{"txn_id": 2, "op": "BEGIN"}
{"txn_id": 2, "op": "UPDATE", "table": "medicines", "key": 5,
 "value": {"stock": 39, ...}, "old_value": {"stock": 40, ...}}
{"txn_id": 2, "op": "UPDATE", "table": "patients",  "key": 5,
 "value": {"balance": 380, ...}, "old_value": {"balance": 500, ...}}
{"txn_id": 2, "op": "INSERT", "table": "prescriptions",
 "key": 401, "value": {"prescription_id": 401, ...}}
{"txn_id": 2, "op": "COMMIT"}
```


B Module A ACID Property Matrix

Table 17: Module A – How each ACID property is implemented

Property	Implementation	Verified By
Atomicity	LIFO undo log replayed on ROLLBACK; all-or-nothing B+ Tree writes	`test_atomicity_rollback`
Consistency	Application-layer constraint checks before B+ Tree writes	`test_consistency`
Isolation	Exclusive row-level locks via `LockManager`; `LockConflictError` on contention	`test_isolation`
Durability	WAL fsync before every write; JSON serialisation on COMMIT; REDO-only crash recovery	`test_durability`

C Module B RBAC Endpoint Access Matrix

Table 18: Endpoint access summary across all roles (from API_DOCS_TO_RUN.md)

Endpoint	SuperAdm	Admin	Doctor	Faculty	Nurse	Pharma	Staff	Support	Tech
`GET /members`	✓	✓	R	Own	R	R	Own	R	R
`POST /members`	✓	✓	×	×	×	×	×	×	×
`DELETE /members`	✓	×	×	×	×	×	×	×	×
`GET /appointments`	✓	✓	Own	Own	✓	✓	Own	✓	✓
`POST /prescriptions`	✓	✓	✓	×	×	×	×	×	×
`GET /inventory`	✓	✓	×	×	R	✓	×	R	R
`GET /billing`	✓	✓	×	×	×	✓	×	×	×
`GET /visits`	✓	✓	Own	×	✓	×	×	R	R
`GET /staff`	✓	✓	×	×	×	×	×	×	×

✓ = Full access R = Read-only Own = Own records only × = Blocked